

Condensed Coursework Chat Transcript

User:

Files mentioned by the user:

tomila.docx: /Users/user/Downloads/tomila.docx

My request for Codex:

I need this coursework to be done strictly according to the instructions provided in the file. Please help

Assistant:

The coursework package is now set up in the workspace and follows the brief from `tomila.docx` as closely as we can automate. The main app is in [app.py](/Users/user/uni/tomila%20ai/app.py), with required docs in [README.md](/Users/user/uni/tomila%20ai/README.md), [AI_USAGE.md](/Users/user/uni/tomila%20ai/AI_USAGE.md), [PROMPTS_USED.md](/Users/user/uni/tomila%20ai/PROMPTS_USED.md), and demo data in [sample_data/retail_sales_dirty.csv](/Users/user/uni/tomila%20ai/sample_data/retail_sales_dirty.csv) and [sample_data/hr_records_dirty.json](/Users/user/uni/tomila%20ai/sample_data/hr_records_dirty.json). I also added [outputs/transformation_report_example.csv](/Users/user/uni/tomila%20ai/outputs/transformation_report_example.csv), [DEPLOYMENT_URL.txt](/Users/user/uni/tomila%20ai/DEPLOYMENT_URL.txt), and [FINAL_REPORT_MANUAL_ONLY.md](/Users/user/uni/tomila%20ai/FINAL_REPORT_MANUAL_ONLY.md).

The app includes the 4 required pages: upload/overview, cleaning/preparation, visualization builder, and export/report. It supports CSV/XLSX/JSON upload, missing-value handling, duplicates, type conversion, categorical cleaning, outlier handling, scaling, column operations, validation rules, 6 matplotlib chart types, export of cleaned data, and a reproducible transformation log with undo/reset.

Verification: `python3 -m py\_compile app.py` passed, and the sample datasets meet the minimum size requirement with 1520 rows / 10 columns and 1212 rows / 10 columns. I could not fully run Streamlit here because the dependencies are not installed in this environment yet.

Two important manual items are still yours to complete because the brief requires them outside AI-generated work:

- deploy the app and put the live link into

[DEPLOYMENT_URL.txt](/Users/user/uni/tomila%20ai/DEPLOYMENT_URL.txt)

- write the final 2-page report manually without AI, as noted in [FINAL_REPORT_MANUAL_ONLY.md](/Users/user/uni/tomila%20ai/FINAL_REPORT_MANUAL_ONLY.md)

If you want, I can next help you with the final polish step: tightening the UI, preparing a demo-video walkthrough, or checking the app feature-by-feature against the rubric.

User:

Context from my IDE setup:

Open tabs:

- requirements.txt: requirements.txt
- DEPLOYMENT_URL.txt: DEPLOYMENT_URL.txt
- PROMPTS_USED.md: PROMPTS_USED.md
- AI_USAGE.md: AI_USAGE.md

- app.py: app.py

My request for Codex:

let's set up the environment and test it out

Assistant:

The environment is set up and the app passed a basic smoke test.

I created `.venv`, installed the dependencies from

[requirements.txt](/Users/user/uni/tomila%20ai/requirements.txt), and verified the key imports work. Then I launched the app from [app.py](/Users/user/uni/tomila%20ai/app.py) with Streamlit, confirmed it started on

`http://127.0.0.1:8501`, and checked that the server responded with `HTTP/1.1 200 OK`.

To run it again locally:

```
```bash
cd "/Users/user/uni/tomila
ai" source .venv/bin/activate
streamlit run app.py
```
```

Two notes from testing:

- Installing packages needed network access, so that part is now done.
- Starting the Streamlit server needed running outside the sandbox, but it worked once allowed.

If you want, I can next do a real feature test with one of the sample datasets and fix any UI or logic issues we find.

User:

Context from my IDE setup:

Open tabs:

- hr_records_dirty.json: sample_data/hr_records_dirty.json
- requirements.txt: requirements.txt
- DEPLOYMENT_URL.txt: DEPLOYMENT_URL.txt
- PROMPTS_USED.md: PROMPTS_USED.md
- AI_USAGE.md: AI_USAGE.md

My request for Codex:

why top for manager_is null tho?

User:

Context from my IDE setup:

Active file: README.md

Open tabs:

- README.md: README.md
- PROMPTS_USED.md: PROMPTS_USED.md
- FINAL_REPORT_MANUAL_ONLY.md: FINAL_REPORT_MANUAL_ONLY.md
- transformation_report_example.csv: outputs/transformation_report_example.csv

- app.py: app.py

My request for Codex:
let's implement it then

Assistant:

Google Sheets support is now implemented in [app.py](/Users/user/uni/tomila%20ai/app.py).

On Page A, there's now an `Optional Google Sheets URL` field plus a `Load Google Sheet` button. The app accepts a normal Google Sheets tab URL, converts it into a CSV export link, loads that tab into pandas, and sends it through the same type-inference pipeline as uploaded files. I also made sure both common URL styles work:

- `.../edit#gid=123`
- `.../edit?gid=123`

I updated [README.md](/Users/user/uni/tomila%20ai/README.md) too, so the bonus feature is documented for your submission/demo.

Verification:

- `python3 -m py\_compile app.py` passed
- the URL conversion helper was tested successfully for both fragment-based and query-based `gid` formats

One practical note: this works best with a public or properly shared Google Sheet, because the deployed app needs permission to access the sheet's CSV export URL.

If you want, I can do the next step and run a live test with a real Google Sheets link or tighten the UI wording so it looks more polished for submission.

User:

Context from my IDE setup:

Active file: sample_data/hr_records_dirty.json

Open tabs:

- hr_records_dirty.json: sample_data/hr_records_dirty.json
- README.md: README.md
- PROMPTS_USED.md: PROMPTS_USED.md
- FINAL_REPORT_MANUAL_ONLY.md: FINAL_REPORT_MANUAL_ONLY.md
- transformation_report_example.csv: outputs/transformation_report_example.csv

My request for Codex:
can we combine both rules? I think it would be logical

User:

Context from my IDE setup:

Active file: sample_data/hr_records_dirty.json

Open tabs:

- hr_records_dirty.json: sample_data/hr_records_dirty.json

- README.md: README.md
- PROMPTS_USED.md: PROMPTS_USED.md
- FINAL_REPORT_MANUAL_ONLY.md: FINAL_REPORT_MANUAL_ONLY.md
- transformation_report_example.csv: outputs/transformation_report_example.csv

My request for Codex:

can we make a button to download the plot

User:

Context from my IDE setup:

Active file: PROMPTS_USED.md

Open tabs:

- PROMPTS_USED.md: PROMPTS_USED.md

My request for Codex:

Medium: the coursework asks for a mapping dictionary via a UI table editor, but the app currently uses a free-text textarea instead. See app.py (line 415). That may still work functionally, but it is not a strict match to the wording.

Medium: the brief's formula examples include `log(col)`, but the current formula feature uses `DataFrame.eval`, so examples like that are likely to fail unless we explicitly support math functions. See app.py (line 511).

we need to implement these two above then, if it is required per instructions.

Low: Google Sheets is implemented as shareable-URL import, but the brief phrases it as "via service account or OAuth." Since it is optional, this does not block the core coursework, but for claiming the bonus I'd describe it carefully as lightweight Google Sheets support rather than full authenticated integration.

is setting up service account or OAuth difficult?

and then we will proceed with the AI feature you suggested

Assistant:

The two strict-brief fixes are done in [app.py](/Users/user/uni/tomila%20ai/app.py).

The categorical mapping now uses a real editable table via `st.data_editor` instead of a free-text box, which is much closer to the brief's "UI table editor" wording. The formula feature now supports expressions like `log(sales_amount)`, `profit - profit.mean()`, and `sales_amount.fillna(0)` through a controlled evaluator instead of plain `DataFrame.eval`. I also verified both changes locally.

On Google Sheets auth: yes, "service account" or "OAuth" is noticeably more work than the shareable-URL approach.

- "Service account" is easier than OAuth for development, but only works well if the sheet is explicitly shared with the service account email.
- "OAuth" is the heaviest option because you need Google Cloud setup, consent screen, client credentials, redirect handling, and token flow.
- For this coursework, since Google Sheets is optional and bonus-only, I would not do service

account/OAuth unless you specifically want to chase the extra polish or bonus claim very strictly.

So my recommendation is:

- keep the current lightweight Google Sheets support as-is
- move next to the optional AI bonus feature

The easiest AI feature for this codebase is still chart suggestion. It's the lowest-risk one because it only recommends visualization settings and can use a confirmation step before applying anything. If you want, I can implement that next.

User:

Context from my IDE setup:

Active file: FULL_CHAT_TRANSCRIPT.json

Active selection of the file:

both f

Open tabs:

- FULL_CHAT_TRANSCRIPT.json: FULL_CHAT_TRANSCRIPT.json
- hr_records_dirty.json: sample_data/hr_records_dirty.json
- PROMPTS_USED.md: PROMPTS_USED.md

My request for Codex:

let's make that AI feature

Assistant:

The AI feature is in.

What I added:

- an optional `Optional AI Assistant` section on Page C in [app.py](/Users/user/uni/tomila%20ai/app.py)
- `Enable AI assistant` toggle
- plain-language chart request box
- `Suggest chart with AI` button
- confirmation step with `Apply AI suggestion`
- OpenAI-backed suggestion flow using `OPENAI\_API\_KEY`
- docs update in [README.md](/Users/user/uni/tomila%20ai/README.md)
- dependency update in [requirements.txt](/Users/user/uni/tomila%20ai/requirements.txt)

How it works:

- the app sends dataset schema + your chart goal to OpenAI
- it gets back a suggested `plot\_type`, `x`, `y`, `group`, `aggregation`, and `title`
- it shows you the suggestion first
- only after you click `Apply AI suggestion` does it change the chart controls

That matches the brief pretty well:

- optional toggle
- works without AI
- confirmation workflow
- clearly labeled as potentially imperfect

What I could verify:

- the code compiles cleanly
- the UI and config flow are wired in

What I could not fully verify here:

- live OpenAI response behavior, because there is no API key configured in this environment

To test it, add `OPENAI\_API\_KEY` locally or in Streamlit secrets, then reinstall requirements if needed:

```
```bash
cd "/Users/user/uni/tomila
ai" source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py
```
```

If you want, next I can:

- add a `.streamlit/secrets.toml.example` file for deployment
- make the transcript export cleaner
- or help you prepare the GitHub + Streamlit deployment now

User:

Context from my IDE setup:

Active file: sample_data/hr_records_dirty.json

Open tabs:

- hr_records_dirty.json: sample_data/hr_records_dirty.json
- FULL_CHAT_TRANSCRIPT.json: FULL_CHAT_TRANSCRIPT.json
- PROMPTS_USED.md: PROMPTS_USED.md

My request for Codex:

where do I add my openai key?